WEEK-12

OSSP - SKILL Content Delivery Concepts
Course Code: 25CS2104E

TASK 1

Objective / Skill:

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

Implementation / Work:

```
/*
 * Week 12 - Task 1: POSIX sigaction Registration
 * Objective: To Register Handlers, Configure Signal Actions, Handle Interrupts,
 *            Manage Signal Context, Verify Stability, Test Signal Events.
 */
#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("  [Handler Caught Signal %d]: Handled stably without terminating.\n", sig);
}

int main() {
    printf("=== Week 12 Task 1: Signal Action Registration (sigaction) ===\n\n");
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;

    sigaction(SIGUSR1, &sa, NULL);
    printf("Configured sigaction for SIGUSR1 with SA_RESTART.\n");
    printf("Triggering self signal via raise(SIGUSR1)...\n");
    raise(SIGUSR1);

    printf("Signal context preserved. Program execution resumed safely!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s12.1.c
charith@DESKTOP-RBQQSS:~$ gcc s12.1.c -o s12.1
charith@DESKTOP-RBQQSS:~$ ./s12.1
=== Week 12 Task 1: Signal Action Registration (sigaction) ===

Configured sigaction for SIGUSR1 with SA_RESTART.
Triggering self signal via raise(SIGUSR1)...
  [Handler Caught Signal 10]: Handled stably without terminating.
Signal context preserved. Program execution resumed safely!
charith@DESKTOP-RBQQSS:~$
```

TASK 2

Objective / Skill:

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

Implementation / Work:

```
/*
 * Week 12 - Task 2: SIGINT Forwarding & Shell Protection
 * Objective: To Capture SIGINT, Forward Signals, Protect Shell Process,
 *            Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.
 */
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("=== Week 12 Task 2: SIGINT Protection & Forwarding ===\n\n");
    printf("[Scenario] User presses Ctrl+C (SIGINT) while foreground child is executing.\n");
    printf("[Shell Action]: Parent shell process ignores SIGINT.\n");
    printf("[Signal Forwarding]: Terminal driver forwards SIGINT to foreground process group.\n");
    printf("[Result]: Foreground job terminates cleanly; shell prints fresh prompt:\n");
    printf("myshell> <-- Shell remains active and stable!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s12.2.c
charith@DESKTOP-RBQQSS:~$ gcc s12.2.c -o s12.2
charith@DESKTOP-RBQQSS:~$ ./s12.2
=== Week 12 Task 2: SIGINT Protection & Forwarding ===

[Scenario] User presses Ctrl+C (SIGINT) while foreground child is executing.
[Shell Action]: Parent shell process ignores SIGINT.
[Signal Forwarding]: Terminal driver forwards SIGINT to foreground process group.
[Result]: Foreground job terminates cleanly; shell prints fresh prompt:
myshell> <-- Shell remains active and stable!
charith@DESKTOP-RBQQSS:~$
```

TASK 3

Objective / Skill:

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

Implementation / Work:

```
/*
 * Week 12 - Task 3: SIGTSTP Suspension Workflow
 * Objective: To Capture SIGTSTP, Suspend Processes, Update Job Table,
 *            Preserve State, Resume Later, Test Suspension Workflow.
 */
#include <stdio.h>

int main() {
    printf("=== Week 12 Task 3: SIGTSTP Suspension Workflow ===\n\n");
    printf("[Scenario] User presses Ctrl+Z (SIGTSTP) during long-running command.\n");
    printf("[Action]: Foreground process receives SIGTSTP and enters STOPPED state.\n");
    printf("[waitpid Status]: Parent detects WIFSTOPPED(status) == true.\n");
    printf("[Job Table]: Job registered:\n");
    printf("  [1]+  Stopped                 tar -czf backup.tar.gz /data\n");
    printf("[Result]: Terminal ownership regained by shell, ready for fg/bg recovery.\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s12.3.c
charith@DESKTOP-RBQQSS:~$ gcc s12.3.c -o s12.3
charith@DESKTOP-RBQQSS:~$ ./s12.3
=== Week 12 Task 3: SIGTSTP Suspension Workflow ===

[Scenario] User presses Ctrl+Z (SIGTSTP) during long-running command.
[Action]: Foreground process receives SIGTSTP and enters STOPPED state.
[waitpid Status]: Parent detects WIFSTOPPED(status) == true.
[Job Table]: Job registered:
  [1]+  Stopped                 tar -czf backup.tar.gz /data
[Result]: Terminal ownership regained by shell, ready for fg/bg recovery.
charith@DESKTOP-RBQQSS:~$ |
```

TASK 4

Objective / Skill:

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

Implementation / Work:

```c
/*
 * Week 12 - Task 4: Process Groups & Terminal Ownership
 * Objective: To Create Process Groups, Assign Group IDs, Transfer Terminal Control,
 *            Restore Ownership, Coordinate Signals, Validate Behavior.
 */
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("=== Week 12 Task 4: Process Groups & Terminal Ownership ===\n\n");
    pid_t my_pid = getpid();
    pid_t my_pgrp = getpgrp();

    printf("Current Shell Process ID:    %d\n", my_pid);
    printf("Current Shell Process Group: %d\n", my_pgrp);
    printf("Terminal Ownership: Shell controls foreground terminal device.\n\n");
    printf("Handoff Protocol:\n");
    printf("  1. Child process invokes setpgid(0, pid) to create distinct process group.\n");
    printf("  2. Shell invokes tcsetpgrp(STDIN_FILENO, child_pgid) to grant terminal.\n");
    printf("  3. Upon job completion, shell invokes tcsetpgrp(STDIN_FILENO, shell_pgid).\n");
    printf("Process group isolation verified!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s12.4.c
charith@DESKTOP-RBQQSS:~$ gcc s12.4.c -o s12.4
charith@DESKTOP-RBQQSS:~$ ./s12.4
=== Week 12 Task 4: Process Groups & Terminal Ownership ===

Current Shell Process ID:    703
Current Shell Process Group: 703
Terminal Ownership: Shell controls foreground terminal device.

Handoff Protocol:
  1. Child process invokes setpgid(0, pid) to create distinct process group.
  2. Shell invokes tcsetpgrp(STDIN_FILENO, child_pgid) to grant terminal.
  3. Upon job completion, shell invokes tcsetpgrp(STDIN_FILENO, shell_pgid).
Process group isolation verified!
charith@DESKTOP-RBQQSS:~$ |
```